

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра вычислительной техники

ОТЧЁТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
ПО ДИСЦИПЛИНЕ «ПРОГРАММИРОВАНИЕ»

Факультет	ЗО	Преподаватель	Дубков Илья
Группа	ДТ-460а		Сергеевич
Студент	Пантюхин Артём Евгеньевич		
Вариант	9		

Цель работы

Изучить структуру класса, механизм создания и использования, описание членов-данных класса и методов доступа к ним, возможность инициализации объектов класса с помощью конструкторов и уничтожение их с помощью деструкторов.

Задание

Реализовать класс в соответствии с вариантом.

Класс должен обеспечивать набор методов для работы с данными. Создать перегруженные конструкторы: конструктор с параметрами, конструктор копирования, конструктор по умолчанию.

Реализовать указанные классы с динамическим выделением памяти для хранения некоторых полей. Создать деструктор для освобождения памяти. Посмотреть в отладчике, как вызываются конструкторы и деструкторы.

Составить демонстрационную программу. Для реализации демонстрационной программы использовать отдельный модуль. Программу построить с использованием проекта. Посмотреть работу программы в отладчике, обратить внимание на представление данных. Построить программу без отладочной информации. Обратить внимание на размер программы.

Вариант 9: Построить класс для работы с информацией о студенте. Класс должен включать следующие поля: фамилию (динамическое выделение памяти для фамилии), возраст, средний балл. Класс должен обеспечивать простейшие методы для работы с полями класса: получение значений, изменение значений, вывод значений

Исходный код

"student_data.h"

```
#ifndef S3L1_STUDENT_DATA_H
#define S3L1_STUDENT_DATA_H

#include <cstdint>

#define S3L1_STUDENT_DATA_DEFAULT_PRINT_NAME "NULL"

namespace s3l1 {
    class StudentData {
    public:
        StudentData();
        StudentData(const char* last_name, int age = 0, float average_score =
0.0);

        ~StudentData();
        StudentData(const StudentData &sd);
        StudentData(StudentData &&sd);
        StudentData& operator=(const StudentData& sd);
        StudentData& operator=(StudentData &&sd);

        const char* get_last_name();
        int get_age();
        float get_average_score();
        bool set_last_name(const char* name);
        bool set_age(int age);
        bool set_average_score(float score);

        void print_last_name();
        void print_age();
        void print_average_score();
        void print_data();

    private:
        char* _last_name;
        int _age;
        float _average_score;

        bool _is_valid_name(const char* name);
        bool _is_valid_age(const int age);
        bool _is_valid_average(const float average);

        void _copy_string(char** dest, const char* src);
        std::size_t _get_true_size(const char* str);

    };
}

#endif //S3L1_STUDENT_DATA_H
```

"student_data.cpp"

```
#include "student_data.h"

#include <cstdint>
#include <cmath>
#include <iostream>
#include <cstring>

namespace s3l1 {
    const char* StudentData::get_last_name() {
        return _last_name;
    }

    int StudentData::get_age() {
        return _age;
    }

    float StudentData::get_average_score() {
        return _average_score;
    }

    bool StudentData::set_last_name(const char* name) {
        const char* size_ptr = name;
        std::size_t size = 0;

        if (!_is_valid_name(name)) {
            return false;
        }

        _copy_string(&_amp;_last_name, name);

        return true;
    }

    bool StudentData::set_age(int age) {
        if (!_is_valid_age(age)) return false;
        _age = age;
        return true;
    }

    bool StudentData::set_average_score(float score) {
        if (!_is_valid_average(score)) return false;

        _average_score = score;
        return true;
    }

    StudentData::StudentData():
        _last_name(NULL),
        _age(0),
        _average_score(0.0)
    {}

    StudentData::StudentData(const char* last_name, int age, float average_score):
StudentData() {
        set_last_name(last_name);
        set_age(age);
        set_average_score(average_score);
    }

    StudentData::~StudentData() {
```

```

        if (_last_name) {
            delete [] _last_name;
        }
    }

StudentData::StudentData(const StudentData &sd) : StudentData(){
    _age = sd._age;
    _average_score = sd._average_score;
    _copy_string(&_amp;_last_name, sd._last_name);
}

StudentData::StudentData(StudentData &&sd): StudentData() {
    _age = sd._age;
    _average_score = sd._average_score;
    _last_name = sd._last_name;
    sd._last_name = NULL;
}

void StudentData::print_last_name() {
    const char* to_print = _last_name ? _last_name :
S3L1_STUDENT_DATA_DEFAULT_PRINT_NAME;
    std::cout << to_print << std::endl;
}

void StudentData::print_age() {
    std::cout << _age << std::endl;
}

void StudentData::print_average_score() {
    std::cout << _average_score << std::endl;
}

void StudentData::print_data() {
    std::cout << "Student: ";
    print_last_name();
    std::cout << "\tAge: ";
    print_age();
    std::cout << "\tAverage Score: ";
    print_average_score();
}

bool StudentData::_is_valid_name(const char* name) {
    if (!name) {
        return false;
    }
    return (*name != 0);
}

bool StudentData::_is_valid_age(const int age) {
    /*
    понятно, что студент вряд ли будет младенцем или старичком 100+,
    но кто-то, например, может быть зачислен с рождения
    или числиться в базах до смерти,
    и мы не хотим вывалиться в ошибку по недосмотру
    */
    return age >= 0 && age <= 150;
}

/*
система оценки не указана, так что просто проверим что это вообще число,
что оно больше нуля,

```

и далее отдадим всё в руки того, кто будет дёргать за ручки нашего интерфейса.
*/

```
bool StudentData::_is_valid_average(const float average) {
    if (std::isnan(average) || std::isinf(average)) {
        return false;
    }
    return average >= 0.0;
}

void StudentData::_copy_string(char** dest, const char* src) {
    if (!dest) return;
    if (*dest == src) return;

    if (*dest) {
        delete[] *dest;
        *dest = NULL;
    }

    if (src) {
        std::size_t size = _get_true_size(src);
        *dest = new char[size];
        memcpy(*dest, src, size);
        (*dest)[size-1] = 0;
    }
}

std::size_t StudentData::_get_true_size(const char* str) {
    if (!str) return 0;

    const char* start = str;
    while (*(str++));
    return str-start;
}
}
```

"main.cpp"

```
#include "student_data.h"
#include <iostream>

void print_title(const char* title) {
    std::cout << "\033[32m|\033[32m| " << title << " |\033[0m" << std::endl;
}

void print_closing(const char* title) {
    std::cout << "\033[32m|^| " << title << " |^|\033[0m" << std::endl;
}

void print_valid_example(s311::StudentData& data) {
    print_title("Setters example (valid)");
    std::cout << "\tSET: Last name: Zaharov" << std::endl;
    std::cout << "\tSET: Age: 33" << std::endl;
    std::cout << "\tSET: Average score: 68.8" << std::endl << std::endl;

    data.set_last_name("Zaharov");
    data.set_age(33);
    data.set_average_score(68.8);
    data.print_data();

    print_closing("Setters example (valid) end");
}

void print_invalid_example(s311::StudentData& data) {
    print_title("Setters example (invalid)");
    std::cout << "\tSET: Last name: \"\"" << std::endl;
    std::cout << "\tSET: Age: -6" << std::endl;
    std::cout << "\tSET: Average score: -88.5" << std::endl << std::endl;

    data.set_last_name("");
    data.set_age(-6);
    data.set_average_score(-88.5);
    data.print_data();
    print_closing("Setters example (invalid) end");
}

s311::StudentData setup_copy() {
    s311::StudentData source("Ryazantsev", 24, 96.3);
    return s311::StudentData(source);
}

void print_simple_part() {
    s311::StudentData student_simple{};
    print_title("Simple constructor");
    student_simple.print_data();
    print_closing("Simple constructor end");
    print_valid_example(student_simple);
    print_invalid_example(student_simple);
}

void print_various_constructors_part() {
    print_title("Full constructor");
    s311::StudentData student_all_fields("Ryazantsev", 24, 96.3);
    student_all_fields.print_data();
    print_closing("Full constructor end");

    print_title("Full constructor (invalid)");
    s311::StudentData student_incorrect("", -444, -3.2);
```

```

    student_incorrect.print_data();
    print_closing("Full constructor (invalid) end");

    print_title("Copy constructor (Ryazantsev to new)");
    std::cout << "Original initialized out of scope and is deleted by this point"
<< std::endl;
    /*
    Чтобы показать копирование значения, а не pointers,
    вызываю отдельную функцию, по истечении которой исходник удалится.
    */
    s311::StudentData student_copied = setup_copy();
    student_copied.print_data();
    print_closing("Copy constructor end");

    print_title("Move constructor (From Ryazantsev)");
    s311::StudentData student_moved = std::move(student_all_fields);
    std::cout << "Destination: " << std::endl;
    student_moved.print_data();
    std::cout << "Source: (will be deleted after function's return)" << std::endl;
    student_all_fields.print_data();
    print_closing("Move constructor end");
}

void print_getters() {
    print_title("Getters example");
    s311::StudentData student("Ryazantsev", 24, 96.3);
    student.print_data();
    std::cout << std::endl;

    std::cout << "student.get_last_name(): " << student.get_last_name() <<
std::endl;
    std::cout << "student.get_age(): " << student.get_age() << std::endl;
    std::cout << "student.get_average_score(): " << student.get_average_score() <<
std::endl;
    print_closing("Getters example end");
}

void print_prints() {
    print_title("Print functions example");

    s311::StudentData student("Ryazantsev", 24, 96.3);

    std::cout << "student.print_data():" << std::endl;
    student.print_data();
    std::cout << "student.print_last_name():" << std::endl;
    student.print_last_name();
    std::cout << "student.print_age():" << std::endl;
    student.print_age();
    std::cout << "student.print_average_score():" << std::endl;
    student.print_average_score();

    print_closing("Print functions end");
}

int main(void) {
    print_simple_part();
    print_various_constructors_part();
    print_getters();
    print_prints();
    return 0;
}

```


Выводы

По результатам выполнения задания, был реализован класс с информацией о студенте. Получено представление о работе конструкторов и деструкторов, представлении классов в памяти компьютера. Изучены модификаторы доступа классов, разница между конструктором копирования и конструктором присваивания.